

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Facultad de Ingeniería — Ingeniería de Sistemas e Informática

Arquitectura Orientada al Servicio

AVANCE DE PROYECTO FINAL 2 (APF2)

Diseño Arquitectónico — Semana 10

“Diseño e Implementación de una Plataforma Digital Multiplataforma para la Gestión Integral de Operaciones en Nachopps Restobar – Sullana, 2026”

Autores:

Palacios Ynga, Marcos Roberto
Otero Reto, Juan Pablo
Requena Peña, Kevin Alexander

Docente:

MSc. Saavedra Navarro, Edwin Arnulfo

Piura – Perú, 2026

Índice

1. Introducción
2. Diagrama C4 — Nivel 1: Contexto
3. Diagrama C4 — Nivel 2: Contenedores
4. Inventario de Servicios
5. Descripción Breve de Cada Servicio
6. Contratos Iniciales de los Servicios Principales
7. Matriz de Integraciones
8. Decisiones Arquitectónicas Clave (ADRs)
9. Riesgos Técnicos Identificados
10. Trazabilidad APF1 → APF2
11. Anexo: Código Structurizr DSL

1. Introducción

El presente documento corresponde al Avance de Proyecto Final 2 (APF2) del curso de Arquitectura Orientada a Servicios. Su propósito es transformar el descubrimiento inicial realizado en el APF1 en una propuesta arquitectónica estructurada, visible y defendible para el caso de estudio: la Plataforma Digital de Gestión Integral de Operaciones para Nachopps Restobar, ubicado en Sullana, Piura.

El documento se estructura siguiendo los ocho entregables exigidos por la rúbrica del APF2: diagramas C4 de Contexto y Contenedores, inventario formal de servicios, descripción breve de cada uno, contratos iniciales, matriz de integraciones, decisiones arquitectónicas clave y riesgos técnicos identificados. Adicionalmente, se incluye una sección de trazabilidad que evidencia la evolución coherente desde el APF1, y un anexo con el código fuente del modelo arquitectónico en Structurizr DSL.

Toda la propuesta mantiene la regla de continuidad establecida en el curso: lo entregado en el APF1 no se descarta ni se rehace, se amplía y refina con mayor rigor técnico. Los nueve servicios identificados en el APF1 (Reservas, Mesas, Pedidos, Cuentas, Inventario, Caja, Reportes, Notificaciones e Identidad) se conservan íntegros, y la matriz de integraciones desarrollada en la Semana 8 se consolida como entregable formal del APF2.

2. Diagrama C4 — Nivel 1: Contexto

El diagrama de Contexto representa la Plataforma Nachopps Restobar como un sistema único y muestra sus relaciones con los actores humanos que la utilizan y los sistemas externos con los que se integra. Responde a la pregunta: «¿Quién usa el sistema y con qué se integra?»

2.1 Actores humanos identificados

- Cliente: reserva mesa vía QR, consulta menú y revisa su cuenta.
- Recepcionista: gestiona las reservas presenciales o por teléfono.
- Mesero: toma pedidos y asigna mesas desde la app móvil.
- Cocina: recibe comandas y actualiza el estado de preparación.
- Bar: recibe órdenes de bebidas y actualiza su estado.
- Cajero: registra pagos, realiza arqueos y cierra turno.
- Administrador: gestiona inventario, configura el sistema y consulta reportes operativos.
- Gerencia: consulta reportes consolidados y métricas de negocio.

2.2 Sistemas externos integrados

- Pasarela de Pagos (Niubiz / Culqi / Yape): procesa pagos electrónicos con tarjeta o QR.
- SUNAT: emisión de comprobantes electrónicos (boleta y factura).
- Gateway SMS / WhatsApp: envío de recordatorios de reserva y confirmaciones al cliente.

2.3 Diagrama de Contexto

El diagrama fue generado en Structurizr DSL y puede visualizarse en el playground oficial de Structurizr (<https://playground.structurizr.com/>) utilizando el código fuente provisto en el Anexo del presente documento.

[Insertar aquí la captura de pantalla del diagrama «Contexto» generado en Structurizr Playground]

3. Diagrama C4 — Nivel 2: Contenedores

El diagrama de Contenedores «abre la caja» del sistema y muestra cómo se estructura internamente la Plataforma Nachopps Restobar. Responde a la pregunta: «¿Cómo se estructura la solución por dentro?». La plataforma está organizada en seis capas verticales:

3.1 Capa de Presentación

Conformada por seis aplicaciones cliente, una por cada tipo de consumidor identificado en el APF1:

- App Mesero (Flutter): registro de pedidos y gestión de mesas desde dispositivos móviles.
- Web Cliente QR (React): interfaz pública para reservas y consulta de menú/cuenta.
- Pantalla Cocina (React): KDS para visualizar pedidos pendientes y actualizar su estado.
- Pantalla Bar (React): KDS específico para bebidas.
- POS Caja (Electron + React): aplicación de escritorio para pagos y arqueos.
- Dashboard Administrador (React): panel de reportes, inventario y supervisión.

3.2 Capa de Aplicación

- API Gateway (Kong / NGINX): único punto de entrada al sistema. Centraliza enrutamiento, validación de tokens JWT, rate limiting y logging.

3.3 Capa de Servicios de Negocio

Comprende los siete servicios que encapsulan las capacidades operativas del restobar, clasificados según la tipología SOA del APF1:

- Servicio de Reservas (Entidad)
- Servicio de Mesas (Entidad)
- Servicio de Pedidos (Entidad)
- Servicio de Cuentas (Entidad)
- Servicio de Inventario (Entidad)
- Servicio de Caja (Tarea)
- Servicio de Reportes (Tarea)

3.4 Capa de Servicios de Utilidad

- Servicio de Identidad: autenticación, autorización por rol (RBAC) y auditoría. Emite JWT.
- Servicio de Notificaciones: broker funcional que enruta eventos hacia los actores correspondientes.

3.5 Capa de Mensajería

- Message Broker (RabbitMQ): recibe y distribuye eventos asíncronos siguiendo el patrón publish/subscribe.

3.6 Capa de Datos

Cada servicio gestiona su propia base de datos PostgreSQL, en cumplimiento del principio de autonomía SOA (ADR-002). No se permite que ningún servicio acceda a la base de datos de otro: toda comunicación de datos ocurre vía contrato REST o evento asíncrono.

3.7 Diagrama de Contenedores

[Insertar aquí la captura de pantalla del diagrama «Contenedores» generado en Structurizr Playground]

4. Inventario de Servicios

Se presenta el catálogo formal de los nueve servicios que componen la plataforma, con su identificador único, tipo SOA, capacidad encapsulada, owner funcional, consumidores principales y estado actual.

ID	Servicio	Tipo SOA	Capacidad encapsulada	Owner funcional	Consumidores
SVC-RES-001	Reservas	Entidad	Ciclo de vida de reservas: creación, confirmación, cancelación y consulta de disponibilidad.	Recepción	Cliente (Web QR), Recepcionista
SVC-MES-002	Mesas	Entidad	Estado de mesas (libre/ocupada/reservada) y asignación a cuentas activas.	Operaciones de Sala	Mesero, Servicio Reservas
SVC-PED-003	Pedidos	Entidad	Registro de ítems por mesa, enrutamiento a cocina/bar y seguimiento de estado.	Jefe de Meseros	Mesero, Cocina, Bar, Servicio Inventario
SVC-CTA-004	Cuentas	Entidad	Asociación de pedidos, división de cuenta, generación de ticket y cierre.	Operaciones de Sala	Mesero, Cliente, Servicio Caja
SVC-INV-005	Inventario	Entidad	Control de stock: entradas, descuentos automáticos y alertas de nivel crítico.	Administración / Almacén	Administrador, Servicio Pedidos
SVC-CAJ-006	Caja	Tarea	Orquesta flujo de pago: registra ingresos/egresos, gestiona arqueos y cierre de turno.	Cajero Jefe	Cajero (POS), Admin, Reportes
SVC-REP-007	Reportes	Tarea	Consolida datos de Caja, Pedidos e Inventario para dashboards y reportes.	Gerencia	Administrador, Gerencia
SVC-NOT-008	Notificaciones	Utilidad	Broker funcional: enruta y entrega notificaciones en tiempo real a los actores.	TI / Plataforma	Todos los servicios
SVC-IDE-009	Identidad	Utilidad	Autenticación, autorización por rol (RBAC) y auditoría. Emite y renueva JWT.	TI / Admin del Sistema	Todos los servicios

5. Descripción Breve de Cada Servicio

SVC-RES-001 — Servicio de Reservas

Servicio de entidad responsable del ciclo de vida completo de las reservas: creación, confirmación, cancelación y consulta de disponibilidad horaria. Su boundary excluye el estado físico de las mesas; solo conoce la intención de reserva. Publica los eventos ReservaCreada y ReservaCancelada para que Mesas y Notificaciones reaccionen. Requiere validar la expiración automática de reservas no confirmadas.

SVC-MES-002 — Servicio de Mesas

Servicio de entidad con estado propio (libre/ocupada/reservada). Independiente de Reservas: este servicio sí conoce el estado físico de cada mesa y gestiona la transición de «reservada» a «ocupada» al hacer check-in del cliente. Consume ReservaCreada para marcar mesas como reservadas y publica MesaAsignada y MesaLiberada para sincronizar con el resto del sistema.

SVC-PED-003 — Servicio de Pedidos

Capacidad central del negocio. Registra ítems por mesa y los enruta automáticamente a cocina o bar según el campo «area» del ítem. No incluye lógica de cobro ni descuento de stock. Publica PedidoCreado y PedidoListo; consume MesaAsignada. NFR crítico: latencia ≤ 500 ms en creación de pedido. Trazabilidad completa por ítem.

SVC-CTA-004 — Servicio de Cuentas

Servicio de entidad que agrupa pedidos por mesa y genera el ticket. Sin este servicio no hay cierre de mesa ni trazabilidad del consumo. Consume PedidoListo para incorporar ítems al ticket, publica CuentaCerrada y TicketGenerado. Requiere aclarar si dividirCuenta() genera múltiples tickets o un reparto de montos sobre un único ticket.

SVC-INV-005 — Servicio de Inventario

Servicio de entidad crítico para el control de costos. Controla entradas de stock, descuento automático por pedido y alertas de nivel crítico. Consume CuentaCerrada para descontar stock y publica StockBajo cuando un producto llega al mínimo. Riesgo abierto: definir si el descuento de stock ocurre al crear el pedido o al cerrar la cuenta.

SVC-CAJ-006 — Servicio de Caja

Servicio de tarea que orquesta el flujo de pago. Registra movimientos (ingresos/egresos), realiza arqueos y emite el reporte de cierre de turno. Soporta múltiples medios de pago (efectivo, tarjeta, QR). Consume CuentaCerrada y publica PagoRegistrado y ArqueoRealizado. Requiere consistencia eventual: CuentaCerrada no se pierde si Caja cae (cola persistente + idempotencia).

SVC-REP-007 — Servicio de Reportes

Servicio de tarea que consume eventos de Caja, Pedidos e Inventario para construir un snapshot agregado de operaciones. No almacena datos transaccionales propios; alimenta dashboards y exportaciones. Consume PagoRegistrado, CuentaCerrada y StockDescontado de forma asíncrona. No participa en el camino crítico operativo.

SVC-NOT-008 — Servicio de Notificaciones

Servicio de utilidad. Actúa como broker funcional de eventos: consume todos los eventos del sistema y los enruta a los actores correspondientes (cocina, bar, mesero, cliente vía SMS/WhatsApp). No bloquea el flujo principal: todas sus operaciones son asíncronas.

SVC-IDE-009 — Servicio de Identidad

Servicio de utilidad. Autentica usuarios, autoriza acciones por rol (RBAC) y registra auditoría de acciones críticas. Emite y renueva tokens JWT. NFR crítico: latencia ≤ 200 ms, disponibilidad 99.9% (su caída detiene todo). Riesgo: punto único de falla; mitigación con caché de tokens validados en el API Gateway.

6. Contratos Iniciales de los Servicios Principales

A continuación se presentan las fichas contractuales de los nueve servicios. Las fichas de Pedidos, Caja e Identidad fueron desarrolladas en avances previos (Semana 7 y Semana 8) y se mantienen sin modificación. Las fichas de Reservas, Mesas, Cuentas, Inventario, Reportes y Notificaciones se desarrollan en el presente avance al mismo nivel de detalle.

6.1 Ficha SVC-RES-001 — Servicio de Reservas

ID	SVC-RES-001
Nombre	Servicio de Reservas
Propósito	Gestionar el ciclo de vida completo de las reservas del restobar: creación, confirmación, cancelación y consulta de disponibilidad horaria.
Capacidad asociada	Gestión de reservas
Boundary — Incluye	Crear reserva, confirmar reserva, cancelar reserva, consultar disponibilidad por fecha/hora, expiración automática de reservas no confirmadas.
Boundary — Excluye	Asignar mesa física (lo hace Mesas), cobrar adelanto (lo hace Caja), notificar al cliente (lo hace Notificaciones).
Owner funcional	Recepción
Owner técnico	Equipo Backend – Módulo Sala
Consumidores	Cliente (Web QR), Recepcionista, Servicio Mesas (consume ReservaCreada).
Dependencias	Identidad (JWT), Notificaciones (envío de confirmación al cliente).
Operaciones preliminares	crearReserva(), confirmarReserva(), cancelarReserva(), consultarDisponibilidad()
Tipo de interacción	Mixta: creación/consulta síncrona (REST); ReservaCreada y ReservaCancelada asíncronos (broker).
Datos principales	Reserva {id, clienteld, fecha, hora, mesaPreferida, numComensales, estado, timestamp}
NFR crítico	Latencia ≤ 600 ms en consulta de disponibilidad. Disponibilidad 99.5% en horario operativo. Sin sobrebooking.
Riesgos abiertos	Política de expiración automática de reservas no confirmadas tras X minutos y liberación del slot.
Estado	Diseñado preliminar

Endpoints REST principales:

Endpoint	Método	Propósito	Consumidor
/reservas	POST	Crear nueva reserva	Cliente (QR), Recepcionista
/reservas/{id}/confirmar	PATCH	Confirmar reserva pendiente	Recepcionista
/reservas/{id}	DELETE	Cancelar reserva	Cliente, Recepcionista
/reservas/disponibilidad	GET	Consultar slots disponibles por fecha/hora	Cliente (QR), Recepcionista

6.2 Ficha SVC-MES-002 — Servicio de Mesas

ID	SVC-MES-002
Nombre	Servicio de Mesas
Propósito	Controlar el estado físico de cada mesa (libre/ocupada/reservada) y su asignación a cuentas activas.
Capacidad asociada	Gestión de mesas físicas
Boundary — Incluye	Consultar estado de mesa, asignar mesa a cuenta, liberar mesa, transición de «reservada» a «ocupada» en check-in.
Boundary — Excluye	Crear/confirmar reservas (lo hace Reservas), registrar pedidos (lo hace Pedidos), cobrar (lo hace Caja).
Owner funcional	Operaciones de Sala
Owner técnico	Equipo Backend – Módulo Sala
Consumidores	Mesero (App), Servicio Reservas, Servicio Cuentas.
Dependencias	Identidad (JWT). Sin dependencias funcionales en tiempo de ejecución.
Operaciones preliminares	consultarEstado(), asignarMesa(), liberarMesa()
Tipo de interacción	Mixta: consulta/asignación síncrona (REST); MesaAsignada y MesaLiberada asíncronos. Consume ReservaCreada.
Datos principales	Mesa {id, numero, capacidad, ubicacion, estado, cuentaAsociada, timestamp}
NFR crítico	Latencia ≤ 300 ms en consulta. Consistencia fuerte: una mesa no puede estar asignada a dos cuentas simultáneamente.
Riesgos abiertos	Conflicto si dos meseros intentan asignar la misma mesa simultáneamente. Mitigar con bloqueo optimista.
Estado	Diseñado preliminar

Endpoints REST principales:

Endpoint	Método	Propósito	Consumidor
/mesas	GET	Listar todas las mesas con su estado actual	Mesero, Recepcionista
/mesas/disponibles	GET	Listar mesas libres	Mesero
/mesas/{id}/asignar	PATCH	Asignar mesa a una cuenta	Mesero
/mesas/{id}/liberar	PATCH	Liberar mesa tras pago	Mesero, Servicio Caja

6.3 Ficha SVC-PED-003 — Servicio de Pedidos

La ficha contractual del Servicio de Pedidos fue documentada íntegramente en el Avance de Semana 7 (ARqui_S7s1) y se mantiene sin modificación. A continuación se reproduce el resumen consolidado:

ID	SVC-PED-003
Nombre	Servicio de Pedidos
Propósito	Capacidad central del negocio. Registrar ítems por mesa y enrutarlos automáticamente a cocina o bar. No incluye lógica de cobro ni descuento de stock para mantener un boundary definido.
Capacidad asociada	Gestión de pedidos de sala
Boundary — Incluye	Crear pedido, agregar ítems, actualizar estado (pendiente / en preparación / listo), consultar pedido por mesa.
Boundary — Excluye	Calcular totales, descontar stock, registrar pagos, asignar mesas.
Owner funcional	Operaciones de Sala / Jefe de Meseros
Owner técnico	Equipo Backend – Módulo Sala
Consumidores	Mesero (App), Cocina (Pantalla), Bar (Pantalla), Servicio Inventario (evento PedidoListo).
Dependencias	Mesas (MesaAsignada), Notificaciones (PedidoListo), Identidad (JWT).
Operaciones preliminares	crearPedido(), agregarItem(), actualizarEstado(), consultarPedido()
Tipo de interacción	Mixta: creación/consulta síncrona (REST); PedidoCreado y PedidoListo asíncronos (broker).
Datos principales	Pedido {id, mesald, items[], estado, timestamp, area}; Item {id, nombre, cantidad, area, estado}
NFR crítico	Latencia ≤ 500 ms en crearPedido(). Disponibilidad 99.5% en horario operativo. Trazabilidad completa por ítem.

Riesgos abiertos	Cancelación de pedido ya enviado a cocina: requiere política de cancelación tardía y compensación.
Estado	Diseñado preliminar

Endpoints REST principales (contrato OpenAPI completo desarrollado en Semana 7):

Endpoint	Método	Propósito	Consumidor
/pedidos	POST	Registrar nuevo pedido con sus ítems y enrutar a cocina/bar	Mesero (App), Cliente (QR)
/pedidos/{id}	GET	Consultar detalle y estado actual del pedido	Mesero, Cliente, Caja
/pedidos/{id}/estado	PATCH	Actualizar estado operativo (En preparación → Listo)	Cocina, Bar (KDS)

6.4 Ficha SVC-CTA-004 — Servicio de Cuentas

ID	SVC-CTA-004
Nombre	Servicio de Cuentas
Propósito	Asociar todos los pedidos de una mesa, calcular total, dividir cuenta si aplica, generar el ticket y cerrar la cuenta.
Capacidad asociada	Gestión de cuentas y tickets
Boundary — Incluye	Abrir cuenta por mesa, agregar pedidos a la cuenta, calcular total con descuentos, dividir cuenta, generar ticket, cerrar cuenta.
Boundary — Excluye	Registrar pagos (Caja), descontar stock (Inventario), emitir comprobante fiscal (se delega a SUNAT).
Owner funcional	Operaciones de Sala
Owner técnico	Equipo Backend – Módulo Sala
Consumidores	Mesero (App), Cliente (Web QR), Servicio Caja.
Dependencias	Pedidos (GET /pedidos?mesa={id} — síncrono), Identidad (JWT).
Operaciones preliminares	abrirCuenta(), agregarPedido(), dividirCuenta(), cerrarCuenta(), generarTicket()
Tipo de interacción	Mixta: cierre/generación síncrona; CuentaCerrada y TicketGenerado asíncronos. Consume PedidoListo.
Datos principales	Cuenta {id, mesald, pedidos[], subtotal, descuento, total, estado, ticket}; Ticket {id, cuentaRef, items[], total, fecha}

NFR crítico	Latencia $\leq$ 700 ms en cierre de cuenta. Consistencia con Pedidos: ningún pedido pendiente debe permitir el cierre.
Riesgos abiertos	Definir si dividirCuenta() genera múltiples tickets o un reparto de montos sobre un solo ticket.
Estado	Diseñado preliminar

Endpoints REST principales:

Endpoint	Método	Propósito	Consumidor
/cuentas	POST	Abrir cuenta al asignar mesa	Mesero
/cuentas/{id}	GET	Consultar estado y detalle de cuenta	Mesero, Cliente
/cuentas/{id}/dividir	POST	Dividir cuenta entre comensales	Mesero
/cuentas/{id}/cerrar	POST	Cerrar cuenta y generar ticket	Mesero, Cliente

6.5 Ficha SVC-INV-005 — Servicio de Inventario

ID	SVC-INV-005
Nombre	Servicio de Inventario
Propósito	Controlar el stock de productos: registrar entradas, descontar automáticamente por pedido, mantener stock mínimo y emitir alertas.
Capacidad asociada	Gestión de stock y almacén
Boundary — Incluye	Registrar entrada de stock, consultar disponibilidad, reservar stock al crear pedido, descontar stock al cerrar cuenta, emitir alerta de stock crítico.
Boundary — Excluye	Realizar compras a proveedores (proceso administrativo manual), registrar mermas detalladas (módulo futuro), cobrar (lo hace Caja).
Owner funcional	Administración / Almacén
Owner técnico	Equipo Backend – Módulo Operaciones
Consumidores	Administrador (Dashboard), Servicio Pedidos (consulta y reserva stock), Servicio Reportes (consume StockBajo).
Dependencias	Identidad (JWT).
Operaciones preliminares	registrarEntrada(), consultarStock(), reservarStock(), descontarStock(), actualizarStock()

Tipo de interacción	Mixta: consulta/reserva síncrona (REST); StockBajo asíncrono. Consume CuentaCerrada.
Datos principales	Producto {id, nombre, categoria, stockActual, stockMinimo, unidad, costo}; Movimiento {id, productold, tipo, cantidad, motivo, ts}
NFR crítico	Latencia ≤ 400 ms en consulta de stock. Consistencia eventual para descuentos; síncrona para reservas críticas.
Riesgos abiertos	Definir si el descuento de stock ocurre al crear pedido o al cerrar cuenta. Compromiso entre evitar ventas sin stock y manejar cancelaciones tardías.
Estado	Diseñado preliminar

Endpoints REST principales:

Endpoint	Método	Propósito	Consumidor
/inventario/{productold}	GET	Consultar stock disponible	Servicio Pedidos, Admin
/inventario/reserva	POST	Reservar stock al confirmar pedido	Servicio Pedidos
/inventario/entrada	POST	Registrar entrada de stock	Administrador
/inventario/alertas	GET	Listar productos con stock bajo	Administrador

6.6 Ficha SVC-CAJ-006 — Servicio de Caja

La ficha contractual del Servicio de Caja fue documentada íntegramente en el Avance de Semana 8 (Matriz de Integración) y se mantiene sin modificación. A continuación se reproduce el resumen consolidado:

ID	SVC-CAJ-006
Nombre	Servicio de Caja
Propósito	Orquestar el flujo completo de pago: registrar ingresos/egresos, gestionar arquezos y emitir reporte de cierre de turno.
Capacidad asociada	Gestión de caja y pagos
Boundary — Incluye	Registrar pago, registrar egreso, realizar arqueo de turno, generar reporte de turno.
Boundary — Excluye	Generar tickets, calcular totales de cuenta, gestionar stock, emitir reportes de gestión.
Owner funcional	Administración / Cajero Jefe
Owner técnico	Equipo Backend – Módulo Financiero

Consumidores	Cajero (POS), Administrador (Dashboard), Servicio Reportes (evento PagoRegistrado).
Dependencias	Cuentas (CuentaCerrada), Notificaciones, Identidad.
Operaciones preliminares	registrarPago(), registrarEgreso(), realizarArqueo(), generarReporteTurno()
Tipo de interacción	Mixta: registrarPago() síncrono; consume CuentaCerrada asíncrono; publica PagoRegistrado asíncrono.
Datos principales	Movimiento {id, tipo, monto, medio, cuentaRef, turnold, ts}; Arqueo {turnold, montoReal, diferencia}
NFR crítico	Consistencia: CuentaCerrada no se pierde si Caja cae (cola persistente + idempotencia). Arqueo sin pérdida.
Riesgos abiertos	Inconsistencia si el pago se registra pero CuentaCerrada nunca llega (fallo del broker). Mitigar con Saga.
Estado	Diseñado preliminar

6.7 Ficha SVC-REP-007 — Servicio de Reportes

ID	SVC-REP-007
Nombre	Servicio de Reportes
Propósito	Consolidar datos transaccionales de Caja, Pedidos e Inventario en snapshots agregados para dashboards y reportes gerenciales.
Capacidad asociada	Inteligencia de negocio y consolidación de datos
Boundary — Incluye	Construir vistas materializadas, generar reportes de ventas, stock, ocupación de mesas y rendimiento por turno. Exportar a PDF/Excel.
Boundary — Excluye	Procesar transacciones operativas (Caja/Pedidos), almacenar datos transaccionales detallados (solo agregados).
Owner funcional	Gerencia
Owner técnico	Equipo Backend – Módulo Analítica
Consumidores	Administrador (Dashboard), Gerencia (Dashboard ejecutivo).
Dependencias	Broker (consume eventos), Identidad (JWT). Sin dependencias síncronas.
Operaciones preliminares	generarReporteVentas(), generarReporteStock(), generarDashboard(), exportarReporte()
Tipo de interacción	Asíncrona en alimentación (consume eventos); síncrona en consulta de reportes ya consolidados.

Datos principales	Snapshot agregado por turno, día, semana y mes. No datos transaccionales.
NFR crítico	Latencia ≤ 2 s en generación de reportes consolidados. Disponibilidad 99% (no crítico para operación). Reportes con retraso ≤ 5 min.
Riesgos abiertos	Inconsistencia si eventos llegan fuera de orden. Mitigar con timestamps y reprocesamiento periódico.
Estado	Diseñado preliminar

Endpoints REST principales:

Endpoint	Método	Propósito	Consumidor
/reportes/ventas	GET	Reporte de ventas por periodo	Admin, Gerencia
/reportes/stock	GET	Reporte de stock y rotación	Admin
/reportes/dashboard	GET	Snapshot agregado para dashboard	Admin, Gerencia
/reportes/exportar	POST	Exportar reporte a PDF/Excel	Admin, Gerencia

6.8 Ficha SVC-NOT-008 — Servicio de Notificaciones

ID	SVC-NOT-008
Nombre	Servicio de Notificaciones
Propósito	Actuar como broker funcional de eventos: consumir todos los eventos del sistema y enrutarlos a los actores correspondientes (UI en tiempo real, SMS, WhatsApp).
Capacidad asociada	Comunicación asíncrona transversal
Boundary — Incluye	Suscribir consumidores a eventos, publicar eventos, enrutar notificaciones a Cocina/Bar (UI), enviar SMS/WhatsApp al cliente, registrar estado de entrega.
Boundary — Excluye	Lógica de negocio (qué notificar y cuándo lo definen los servicios productores).
Owner funcional	TI / Plataforma
Owner técnico	Equipo de Plataforma
Consumidores	Todos los servicios (publican eventos); Cocina, Bar, Mesero, Cliente (reciben notificaciones).
Dependencias	Gateway SMS/WhatsApp (sistema externo), Identidad (JWT).

Operaciones preliminares	publicarEvento(), suscribirseAEvento(), enviarNotificacion(), consultarEstadoEntrega()
Tipo de interacción	Asíncrona obligatoria. Pub/Sub puro.
Datos principales	Mensaje {id, eventoOrigen, destinatario, canal, contenido, estado, intentos, ts}; Suscripción {servicio, evento}
NFR crítico	Throughput ≥ 100 msg/s en pico. Sin pérdida de mensajes (colas persistentes). No bloqueante.
Riesgos abiertos	Fallo del broker detiene la coreografía. Mitigar con dead-letter queues, reintentos automáticos y backup de eventos en disco.
Estado	Diseñado preliminar

6.9 Ficha SVC-IDE-009 — Servicio de Identidad

La ficha contractual del Servicio de Identidad fue documentada íntegramente en el Avance de Semana 8 (Matriz de Integración) y se mantiene sin modificación. A continuación se reproduce el resumen consolidado:

ID	SVC-IDE-009
Nombre	Servicio de Identidad
Propósito	Autenticar usuarios, gestionar autorización por rol (RBAC) y registrar auditoría de acciones críticas.
Capacidad asociada	Seguridad y control de acceso
Boundary — Incluye	Autenticar usuario, autorizar acción por rol, registrar auditoría, gestionar y renovar tokens JWT.
Boundary — Excluye	Gestionar datos de empleados, registrar movimientos de caja, controlar acceso físico al local.
Owner funcional	TI / Administrador del Sistema
Owner técnico	Equipo de Seguridad / Plataforma
Consumidores	Todos los servicios (validación de token JWT en cada solicitud entrante).
Dependencias	BD propia de usuarios/tokens. Sin dependencias funcionales de otros servicios.
Operaciones preliminares	autenticar(), autorizarAccion(), registrarAuditoria(), renovarToken()
Tipo de interacción	Síncrona: auth/authz en tiempo real. Latencia crítica ≤ 200 ms.
Datos principales	Usuario {id, nombre, rol, credenciales}; Token {jwt, expiry, userId, rol}; AuditoriaLog {accion, usuario, servicio, ts}

NFR crítico	Latencia $\leq$ 200 ms. Disponibilidad 99.9% (su caída detiene todo). Tokens cifrados (HS256/RS256).
Riesgos abiertos	Punto único de falla: si Identidad cae, el sistema completo se detiene. Evaluar modo degradado con tokens de larga expiración.
Estado	Diseñado preliminar

7. Matriz de Integraciones

La matriz de integraciones fue desarrollada en la Semana 8 (SOA_S08_MatrizIntegracion) y se consolida formalmente como entregable del APF2. Se compone de cuatro tablas que detallan la clasificación síncrono/asíncrono de cada interacción, el mapa de llamadas Request/Response, el presupuesto de latencia del camino crítico y el análisis de servicios bloqueantes.

7.1 Clasificación síncrono / asíncrono

Paso del flujo	Clasificación	Justificación
Validar cliente (Identidad)	Síncrono obligatorio	No se puede procesar ninguna operación sin autenticar al usuario. El servicio de Identidad valida JWT antes de cualquier acción.
Calcular descuento/promoción (Cuentas/Pedidos)	Síncrono obligatorio	El total del ticket depende del descuento aplicado. El mesero/cliente necesita conocer el monto final antes de confirmar.
Reservar / asignar mesa (Mesas)	Síncrono obligatorio	El mesero necesita confirmación inmediata de disponibilidad para asignar al cliente sin conflictos.
Crear pedido (Pedidos)	Síncrono obligatorio	Confirma la intención de orden; cocina/bar no puede actuar sin un pedido creado y con ID asignado.
Descontar stock (Inventario)	Síncrono discutible	Idealmente síncrono para evitar ventas sin stock, pero puede aceptar un modelo eventual si el inventario se valida previamente.
Cerrar cuenta y generar ticket (Cuentas)	Síncrono obligatorio	El cajero necesita el ticket para cobrar. El cliente no puede irse sin que la cuenta esté cerrada formalmente.
Registrar pago (Caja)	Síncrono obligatorio	Valida que el pago fue procesado correctamente antes de liberar la mesa.
Liberar mesa (Mesas)	Síncrono obligatorio	Dependiente del pago confirmado; la mesa debe quedar disponible inmediatamente para el siguiente turno.
Enviar notificación (Notificaciones)	Mejor asíncrono	Alertas a cocina/bar, confirmaciones por WhatsApp/SMS no deben bloquear el flujo principal. Se procesan por evento.
Actualizar reportes (Reportes)	Mejor asíncrono	Los dashboards y consolidados no son críticos para el cierre inmediato. Se alimentan con eventos publicados.

7.2 Mapa de Llamadas Request/Response

Origen	Destino	Endpoint	Tipo	Respuesta esperada	Error crítico
App/Mesero	API Gateway	POST /auth/token	Síncrono	200 JWT token	401 Credenciales inválidas
API Gateway	Serv. Identidad	POST /auth/validate	Síncrono	200 usuario autorizado	403 Token expirado
API Gateway	Serv. Mesas	GET /mesas/disponibles	Síncrono	200 lista de mesas libres	503 servicio no disponible
API Gateway	Serv. Reservas	POST /reservas	Síncrono	201 reserva creada	409 conflicto de horario
Serv. Pedidos	Serv. Inventario	GET /inventario/{itemId}	Síncrono	200 stock disponible	409 stock insuficiente
Serv. Pedidos	Serv. Inventario	POST /inventario/reserva	Síncrono discutible	201 reserva creada	409 stock insuficiente
Serv. Cuentas	Serv. Pedidos	GET /pedidos?mesa={id}	Síncrono	200 pedidos de la mesa	404 mesa sin pedidos
Serv. Cuentas	API Gateway	POST /cuentas/{id}/cerrar	Síncrono	200 ticket generado	422 cuenta ya cerrada
Serv. Caja	Serv. Cuentas	GET /cuentas/{id}/ticket	Síncrono	200 ticket con total	404 cuenta no encontrada
Serv. Caja	API Gateway	POST /pagos	Síncrono	201 pago registrado	402 pago rechazado
Serv. Pedidos	Serv. Notificaciones	POST /notificaciones/evento	Asíncrono	202 evento encolado	— (no bloquea flujo)
Serv. Caja	Serv. Reportes	Evento: PagoRegistrado	Asíncrono	202 evento recibido	— (no bloquea flujo)

7.3 Presupuesto de latencia

Llamada	Latencia estimada	Secuencial/Paralela	Camino crítico
App/Mesero → API Gateway	30 ms	Secuencial	Sí
API Gateway → Serv. Identidad (validar JWT)	80 ms	Secuencial	Sí

Llamada	Latencia estimada	Secuencial/Paralela	Camino crítico
API Gateway → Serv. Mesas (consultar disponibilidad)	100 ms	Secuencial	Sí
Serv. Pedidos → Serv. Inventario (consultar stock)	120 ms	Paralela	Sí
Serv. Pedidos → Serv. Inventario (reservar stock)	150 ms	Secuencial	Sí
Serv. Cuentas → Serv. Pedidos (obtener ítems)	100 ms	Secuencial	Sí
Serv. Cuentas (cerrar cuenta + generar ticket)	120 ms	Secuencial	Sí
Serv. Caja → Serv. Cuentas (obtener ticket)	80 ms	Secuencial	Sí
Serv. Caja (registrar pago)	150 ms	Secuencial	Sí
Serv. Mesas (liberar mesa tras pago)	80 ms	Secuencial	Sí
Procesamiento interno (lógica de negocio)	100 ms	—	—
Serv. Pedidos → Notificaciones (cocina/bar)	200 ms	Asíncrono	No
Serv. Caja → Reportes (evento PagoRegistrado)	150 ms	Asíncrono	No
TOTAL CAMINO CRÍTICO (secuencial)	1,110 ms	Dentro de presupuesto ($\leq 2,000$ ms)	✓

Con paralelismo (Inventario): se reemplaza la suma de consulta + reserva inventario (270 ms) por $\max(150, 100) = 150$ ms. Total optimizado: ~990 ms.

7.4 Análisis de servicios bloqueantes

Servicio	¿Bloquea flujo?	Riesgo	Mitigación propuesta
Identidad	Sí	Ninguna operación sin JWT válido	Cache de tokens validados (TTL corto); circuit breaker si cae
Mesas	Sí	Sin confirmación, no inicia atención	Timeout 500 ms + respuesta de error clara (409/503)

Servicio	¿Bloquea flujo?	Riesgo	Mitigación propuesta
Pedidos	Sí	Cocina/bar no recibe órdenes sin ID	Retry automático con backoff exponencial; estado 'pendiente' si falla
Inventario	Sí/Parcial	Pueden venderse ítems agotados	Fallback: aprobar pedido y marcar revisión manual; alerta StockBajo
Cuentas	Sí	Sin ticket no se cobra ni libera mesa	Timeout 1,000 ms; reintentar generación hasta 3 veces
Caja	Sí	Sin pago confirmado, mesa no se libera	Timeout + respuesta clara; no liberar mesa sin confirmación
Notificaciones	No debería	Si bloquea, cocina espera	Mover 100% a asíncrono mediante RabbitMQ
Reportes	No	Impacto no crítico	Evento asíncrono; no participa en camino crítico

7.5 Mejoras propuestas

Tipo de mejora	Descripción
Mejora de latencia 1	Ejecutar validación de stock (Inventario) en paralelo con la consulta de ítems del pedido, ya que ambas operaciones son independientes. Ahorro estimado: 100-120 ms.
Mejora de latencia 2	Mover la emisión de comprobante fiscal a un proceso diferido (asíncrono) cuando la regulación lo permita. La mesa queda liberada tras el pago, sin esperar el comprobante. Ahorro: 150-200 ms potenciales.
Reducción de acoplamiento 1	Mover las notificaciones a cocina y bar completamente al patrón de coreografía: Pedidos publica PedidoCreado, y Notificaciones lo consume de forma asíncrona. Pedidos no llama directamente al servicio.
Reducción de acoplamiento 2	Implementar cache de validación JWT en el API Gateway (TTL: 60 segundos). Si el servicio de Identidad cae momentáneamente, el Gateway puede seguir validando tokens recientes.

8. Decisiones Arquitectónicas Clave (ADRs)

A continuación se documentan las siete decisiones arquitectónicas que estructuran la solución, en formato Architecture Decision Record (Título, Estado, Contexto, Decisión, Justificación Técnica, Alternativas Consideradas y Consecuencias).

ADR 001 – Adopción del estilo arquitectónico orientado a servicios (SOA)

Título:

Adopción de Arquitectura Orientada a Servicios (SOA) como estilo arquitectónico base para Nachopps Restobar

Estado:

Aceptada

Contexto:

Nachopps Restobar opera con múltiples dominios funcionales (cocina, sala, caja, almacén, recepción) que tienen ciclos de vida, actores y ritmos de cambio claramente distintos. Los procesos actuales son manuales y cada área opera como un silo aislado sin contratos ni interfaces de integración. Esta realidad genera ineficiencias operativas, errores frecuentes y pérdida de información crítica. Se necesitaba un estilo arquitectónico que reflejara la naturaleza distribuida del negocio y permitiera evolución independiente de cada dominio, sin sacrificar la coherencia del sistema global.

Decisión:

Se adoptó la Arquitectura Orientada a Servicios (SOA) como estilo arquitectónico base. Cada dominio funcional del negocio se modela como un servicio autónomo con su propio contrato, lógica y datos. La integración entre servicios ocurre exclusivamente a nivel de contratos (REST/OpenAPI) y eventos (mensajería), nunca por dependencias internas o acceso compartido a datos.

Justificación Técnica:

- Permite modelar con precisión los dominios reales del negocio: cada uno con sus reglas y su propio ritmo de evolución.
- Aplica los principios SOA de autonomía, contrato estandarizado, bajo acoplamiento, abstracción, reutilización, stateless, descubrimiento y composición.
- Facilita la escalabilidad a futuras sucursales sin rediseño estructural.
- Permite reutilización de servicios transversales (Identidad, Notificaciones) sin duplicar lógica.

Alternativas Consideradas:

- Arquitectura Monolítica
 - Ventajas: Simplicidad de desarrollo inicial y despliegue más sencillo.
 - Desventajas: Dificil escalabilidad, fuerte acoplamiento entre dominios, alto impacto ante fallos parciales, dificultad para evolucionar áreas independientes.
- Microservicios de granularidad excesiva
 - Ventajas: Máxima independencia entre componentes.
 - Desventajas: Sobrecarga operativa desproporcionada para un negocio de un solo local; complejidad innecesaria de monitoreo y orquestación.

Consecuencias:

Positivas:

- Autonomía total entre servicios: la caída de uno no bloquea a los demás.
- Integración mediante contratos versionables, no por implementación.
- Escalabilidad horizontal selectiva (solo los servicios con carga crítica).
- Reutilización efectiva de servicios transversales.

Negativas:

- Mayor complejidad operativa: más componentes que monitorear y desplegar.
- Necesidad de versionado riguroso de contratos desde el inicio.
- Curva de aprendizaje para el equipo en patrones de comunicación distribuida.

ADR 002 – Una base de datos por servicio**Título:**

Adopción del patrón «Base de Datos por Servicio» para garantizar autonomía SOA

Estado:

Aceptada

Contexto:

La autonomía es uno de los principios fundamentales de SOA. Compartir una base de datos entre servicios genera acoplamiento implícito por esquema: cualquier cambio en una tabla puede romper varios servicios simultáneamente, y los servicios pierden la capacidad de evolucionar de forma independiente. Adicionalmente, distintos servicios tienen patrones de acceso diferentes (Pedidos requiere baja latencia en escritura, Reportes requiere lecturas agregadas), lo que dificulta optimizar una BD única.

Decisión:

Cada uno de los 9 servicios del sistema tendrá su propia instancia de base de datos PostgreSQL. Ningún servicio puede acceder directamente a la BD de otro; toda comunicación de datos debe pasar por el contrato del servicio o por eventos.

Justificación Técnica:

- Cumple estrictamente el principio de autonomía de SOA.
- Permite optimizar el esquema y los índices según las necesidades específicas de cada servicio.
- Habilita la escalabilidad independiente: el servicio con mayor demanda puede escalar su BD sin afectar a los demás.
- Aísla fallos: la caída o saturación de una BD no degrada todo el sistema.

Alternativas Consideradas:

- Base de datos compartida con schemas lógicos por servicio
 - Ventajas: Menor costo operativo (una sola instancia), facilita backups consolidados.
 - Desventajas: Acoplamiento de disponibilidad; contención de recursos entre servicios; dificultad para optimizar por dominio.
- Base de datos compartida sin schemas separados
 - Ventajas: Simplicidad inicial, joins triviales entre dominios.
 - Desventajas: Acoplamiento total entre servicios; viola autonomía SOA; cualquier migración afecta a todos.

Consecuencias:**Positivas:**

- Autonomía completa: cada servicio evoluciona su modelo de datos sin coordinar con otros.
- Aislamiento de fallos a nivel de persistencia.
- Posibilidad de elegir motor distinto por servicio si fuera necesario en el futuro.

Negativas:

- Mayor costo operativo: 9 instancias de PostgreSQL que mantener y respaldar.
- Sin joins directos entre dominios: consultas cross-dominio se resuelven por composición o vía Reportes.
- Mayor complejidad para garantizar consistencia distribuida (mitigada por ADR 006).

ADR 003 – API Gateway como único punto de entrada**Título:**

Implementación de un API Gateway centralizado para enrutamiento y políticas transversales

Estado:

Aceptada

Contexto:

El sistema cuenta con 6 aplicaciones cliente (App Mesero, Web Cliente QR, Pantalla Cocina, Pantalla Bar, POS Caja, Dashboard Administrador) que necesitan acceder a 9 servicios distintos. Sin un punto único de entrada, cada cliente debe conocer la topología interna, manejar autenticación por cuenta propia y replicar lógica transversal. Esto incrementa el acoplamiento entre clientes y servicios, y dificulta la evolución de las APIs internas.

Decisión:

Se implementó un API Gateway (Kong/NGINX) como único punto de entrada al sistema. El Gateway centraliza el enrutamiento de solicitudes, la validación de tokens JWT, el rate limiting, el logging y el descubrimiento de servicios. Los clientes solo conocen el endpoint del Gateway, no la topología interna de los servicios.

Justificación Técnica:

- Aplica el principio de descubrimiento de SOA: el Gateway funciona como registro de servicios.
- Centraliza la aplicación de políticas transversales (autenticación, throttling, logging) sin duplicarlas en cada servicio.
- Facilita el versionado de APIs sin romper clientes existentes.
- Permite aplicar cache de tokens JWT validados para soportar operación degradada.

Alternativas Consideradas:

- Acceso directo cliente-servicio sin Gateway
 - Ventajas: Menor latencia (un salto menos).
 - Desventajas: Los clientes deben conocer la topología interna; políticas duplicadas; fuerte acoplamiento.
- Backend for Frontend (BFF) por tipo de cliente
 - Ventajas: APIs optimizadas por tipo de consumidor.
 - Desventajas: Mayor cantidad de componentes que mantener; duplicación de lógica de orquestación.

Consecuencias:

Positivas:

- Los clientes no conocen ni dependen de la topología interna.
- Punto único para aplicar políticas de seguridad, rate limiting y observabilidad.
- Facilita el versionado y la evolución de las APIs internas.
- Habilita modo degradado mediante caché de tokens.

Negativas:

- Punto único de falla potencial: requiere alta disponibilidad y configuración redundante.
- Latencia adicional de aproximadamente 30 ms por petición.
- Requiere infraestructura adicional para su despliegue y monitoreo.

ADR 004 – Comunicación híbrida síncrona y asíncrona**Título:**

Adopción de comunicación mixta REST/HTTPS para flujos críticos y RabbitMQ/AMQP para eventos asíncronos

Estado:

Aceptada

Contexto:

El sistema presenta dos tipos claros de interacciones: operaciones que requieren confirmación inmediata para que el usuario pueda continuar (validar stock, registrar pago, asignar mesa) y operaciones que pueden ocurrir en segundo plano sin bloquear el flujo principal (notificar a cocina, alimentar reportes, registrar auditorías). Forzar todo a ser síncrono saturaría el camino crítico; forzar todo a ser asíncrono comprometería la experiencia del usuario.

Decisión:

Se adoptó un modelo de comunicación híbrido: REST/HTTPS para todas las operaciones síncronas del camino crítico, y RabbitMQ con patrón Publish/Subscribe (AMQP) para todos los eventos asíncronos. La clasificación de cada interacción fue documentada en la matriz de integraciones (Sección 7).

Justificación Técnica:

- El camino crítico del flujo de cobro fue cuantificado en 1,110 ms, dentro del presupuesto de 2,000 ms.
- Las operaciones no críticas (notificaciones, reportes) no bloquean el flujo principal.
- La coreografía basada en eventos desacopla productores y consumidores.
- Permite recuperación ante fallos mediante reintentos y dead-letter queues.

Alternativas Consideradas:

- Solo REST síncrono entre todos los servicios
 - Ventajas: Modelo simple y uniforme.
 - Desventajas: Acoplamiento temporal entre servicios; riesgo de fallos en cascada; latencia acumulada excede presupuesto crítico.
- Solo eventos asíncronos (event-driven puro)
 - Ventajas: Máximo desacoplamiento.
 - Desventajas: Operaciones que requieren confirmación inmediata se vuelven complejas; mala experiencia para flujos donde el usuario espera respuesta.

Consecuencias:**Positivas:**

- Camino crítico cumple el presupuesto de latencia ($\leq 2,000$ ms).
- Notificaciones y reportes desacoplados del flujo principal.
- Coreografía permite agregar nuevos consumidores de eventos sin tocar productores.

Negativas:

- Necesidad de implementar patrones para garantizar consistencia eventual (Saga, idempotencia).
- Mayor complejidad operativa al monitorear dos canales de comunicación distintos.
- Requiere infraestructura adicional (RabbitMQ con alta disponibilidad).

ADR 005 – Autenticación basada en JWT con caché en API Gateway**Título:**

Implementación de JSON Web Tokens (JWT) con caché de validación en el Gateway para modo degradado

Estado:

Aceptada

Contexto:

El Servicio de Identidad valida cada solicitud que entra al sistema (NFR: latencia ≤ 200 ms, disponibilidad 99.9%). Si Identidad cae, ninguna operación puede validarse y el sistema completo se detiene. Adicionalmente, validar cada token contra el servicio en cada solicitud añade latencia al camino crítico (~80 ms por validación). Se necesitaba un mecanismo de autenticación resiliente, escalable y rápido, sin sacrificar la seguridad.

Decisión:

Se implementó JWT como mecanismo principal de autenticación y autorización. El Servicio de Identidad emite tokens firmados (HS256/RS256) que los servicios pueden validar localmente mediante la firma. Adicionalmente, el API Gateway mantiene un caché de tokens válidos con TTL corto (60 segundos), de modo que si Identidad cae momentáneamente, el Gateway puede seguir validando solicitudes con tokens recientes.

Justificación Técnica:

- Los tokens JWT son auto-contenidos y verificables sin consultas al backend.
- Reduce la dependencia temporal con el Servicio de Identidad.
- La caché en Gateway desacopla el camino crítico de la disponibilidad puntual de Identidad.
- Estándar ampliamente soportado y compatible con la mayoría de frameworks.

Alternativas Consideradas:

- Sesiones con almacenamiento centralizado (Redis)
 - Ventajas: Control centralizado, revocación inmediata.
 - Desventajas: Acoplamiento al store de sesiones; cada validación requiere acceso a Redis; no escala bien.
- OAuth2 con servidor de autorización externo
 - Ventajas: Alta robustez, estandarización completa.
 - Desventajas: Complejidad excesiva para un sistema de un solo local; sobrecarga operativa desproporcionada.

Consecuencias:**Positivas:**

- Validación descentralizada: cada servicio valida tokens localmente.
- Desacoplamiento del camino crítico respecto a Identidad (gracias a la caché).
- Reducción de latencia: ~80 ms ahorrados en validaciones cacheadas.

Negativas:

- Revocación de tokens no es inmediata: hay que esperar al vencimiento del TTL de la caché (60 s).
- Requiere proteger las claves de firma con extremo cuidado.
- Necesidad de definir política clara de expiración y renovación de tokens.

ADR 006 – Patrón Saga para consistencia distribuida en flujo de pago**Título:**

Aplicación del patrón Saga con compensaciones para garantizar consistencia eventual en el flujo de cobro

Estado:

Aceptada

Contexto:

El flujo de cierre de cuenta y pago involucra a varios servicios autónomos en secuencia: Cuentas (genera ticket) → Caja (registra pago) → Mesas (libera mesa) → Inventario (descuenta stock). Como cada servicio tiene su propia BD (ADR 002) y no existe una transacción distribuida tradicional, una falla parcial podría dejar el sistema en estado inconsistente: por ejemplo, un pago registrado en Caja pero la mesa nunca liberada, o stock no descontado tras un pago confirmado.

Decisión:

Se adoptó el patrón Saga con compensaciones para el flujo crítico de cobro. Cada paso del flujo publica un evento que dispara el siguiente paso. En caso de fallo en cualquier punto, se ejecutan acciones compensatorias en orden inverso para revertir los efectos parciales y dejar el sistema en estado coherente.

Justificación Técnica:

- No requiere transacciones distribuidas (XA, 2PC), no viables en una arquitectura SOA con BD independientes.
- Garantiza consistencia eventual del sistema mediante compensaciones bien definidas.
- Aprovecha la mensajería asíncrona ya implementada (ADR 004).
- Permite trazabilidad completa del flujo, paso por paso.

Alternativas Consideradas:

- Transacciones distribuidas 2PC (Two-Phase Commit)
 - Ventajas: Consistencia fuerte garantizada.
 - Desventajas: Bloqueante, no escalable, requiere coordinador transaccional; no aplica con BD heterogéneas y servicios autónomos.
- Consistencia eventual sin compensaciones (best-effort)
 - Ventajas: Simplicidad inicial.

- Desventajas: Estados inconsistentes silenciosos; auditoría compleja; impacto directo en el negocio.

Consecuencias:**Positivas:**

- Consistencia eventual garantizada sin transacciones distribuidas.
- Trazabilidad completa de cada paso del flujo.
- Compensaciones permiten recuperación automática ante fallos.

Negativas:

- Mayor complejidad en el diseño y prueba: cada operación requiere su compensación correspondiente.
- Necesidad de garantizar idempotencia en cada paso para evitar duplicados ante reintentos.
- Requiere documentación rigurosa de los flujos de compensación.

ADR 007 – Notificaciones como broker funcional de eventos**Título:**

Centralización de la lógica de notificaciones en un servicio de utilidad consumidor de todos los eventos

Estado:

Aceptada

Contexto:

El sistema requiere enviar notificaciones en tiempo real a múltiples actores: cocina y bar (pedidos listos), mesero (pedido listo para servir), cliente (confirmación de reserva por SMS/WhatsApp), administrador (alertas de stock bajo). Si cada servicio implementara su propia lógica de notificación, se duplicaría código, se acoplaría cada servicio a los canales de comunicación (SMS, UI, WhatsApp) y se violaría el principio DRY. Adicionalmente, los servicios de negocio no deberían bloquear su flujo esperando que las notificaciones se envíen.

Decisión:

Se diseñó el Servicio de Notificaciones como un servicio de utilidad que actúa como broker funcional de eventos. Consume todos los eventos publicados por los servicios del sistema y se encarga de enrutarlos al canal apropiado (UI en tiempo real, SMS, WhatsApp). Los servicios productores publican eventos sin saber quién o cómo los consumirá.

Justificación Técnica:

- Aplica el principio de reutilización de SOA: una sola lógica de notificación para todo el sistema.
- Aplica DRY: ningún servicio implementa su propia lógica de notificación.
- Los servicios productores quedan completamente desacoplados de los canales de notificación.
- Permite añadir nuevos canales (push, email, voz) sin tocar los servicios de negocio.

Alternativas Consideradas:

- Cada servicio implementa sus propias notificaciones
 - Ventajas: Lógica de notificación próxima al dominio del servicio.
 - Desventajas: Duplicación de código (DRY violado); acoplamiento de cada servicio a SMS/WhatsApp/UI; mantenimiento disperso.
- Notificaciones integradas dentro del API Gateway

- Ventajas: Centralización en infraestructura ya existente.
- Desventajas: El Gateway se convierte en un God Object; mezcla responsabilidades de enrutamiento con lógica de negocio.

Consecuencias:

Positivas:

- Lógica de notificación centralizada y reutilizable.
- Servicios productores totalmente desacoplados de los canales.
- Facilidad para agregar nuevos canales o cambiar proveedores externos.
- Auditoría centralizada de qué se notificó y cuándo.

Negativas:

- Notificaciones se convierte en componente crítico para la experiencia del usuario.
- Requiere garantizar alta disponibilidad del broker (RabbitMQ) para que los eventos lleguen.
- Una falla del servicio genera retrasos perceptibles en cocina/bar (mitigado con colas persistentes).

9. Riesgos Técnicos Identificados

A continuación se consolidan los riesgos técnicos del proyecto, ampliando los identificados en el APF1 con riesgos específicos del diseño arquitectónico desarrollado en el APF2. Se clasifican por impacto y probabilidad, con su mitigación correspondiente.

ID	Riesgo	Impacto	Probabilidad	Mitigación
RT-01	Definición incorrecta de límites de servicio (muy amplios o muy finos)	Alto	Alta	Aplicar Domain-Driven Design para validar cohesión. Revisión por pares en cada iteración.
RT-02	Contratos mal definidos que generen acoplamiento implícito entre servicios	Alto	Media	Formalizar contratos en OpenAPI y aplicar versionado semántico desde el inicio.
RT-03	Inconsistencia de datos en flujos distribuidos (pago registrado pero stock no descontado)	Alto	Media	Patrón Saga con compensaciones transaccionales en flujos críticos (ADR-006).
RT-04	Fallo del broker de mensajes (RabbitMQ) afectando la coreografía	Alto	Baja	Colas persistentes, reintentos automáticos y dead-letter queues.
RT-05	Servicio de Identidad como punto único de falla	Alto	Media	Caché JWT en Gateway (ADR-005). Modo degradado con tokens de larga expiración.
RT-06	Latencia del camino crítico excede presupuesto en horas pico	Medio	Media	Monitoreo continuo. Paralelización de llamadas independientes (Inventario + Pedidos).
RT-07	Conectividad limitada en el local (Sullana)	Medio	Media	Servicios críticos operan localmente con sincronización diferida a la nube al reconectarse.
RT-08	Conflicto de concurrencia al asignar mesas simultáneamente	Medio	Baja	Bloqueo optimista con versionado en la BD de Mesas.
RT-09	Reservas no confirmadas saturan el calendario	Medio	Media	Política de expiración automática (15 min sin confirmación = liberación).
RT-10	Resistencia del personal al uso del sistema	Medio	Alta	Interfaces simples por rol; capacitación progresiva por módulo.
RT-11	Pasarela de pagos externa caída	Alto	Baja	Fallback a registro manual de pago en efectivo. Sincronización posterior.
RT-12	Pérdida de eventos en RabbitMQ por sobrecarga	Alto	Baja	Configurar persistencia obligatoria + alta disponibilidad en cluster del broker.

10. Trazabilidad APF1 → APF2

Conforme a la regla de continuidad establecida en la rúbrica del curso, el presente avance no descarta ni rehace lo entregado en el APF1; lo amplía, refina y formaliza con mayor rigor técnico. La siguiente tabla evidencia la evolución del trabajo entre ambos hitos.

Aspecto	APF1 (Semana 5)	APF2 (Semana 10)
Servicios identificados	9 servicios candidatos con tipo SOA	Inventario formal con IDs (SVC-XXX-NNN), owners y consumidores específicos
Límites de servicio	Datos, operaciones y mensajes a alto nivel	Boundary «Incluye/Excluye» explícito; NFRs cuantificados; riesgos por servicio
Contratos	Sin contratos formales	Endpoints REST detallados + OpenAPI para Pedidos; fichas técnicas de los 9 servicios
Integraciones	Mensajes publicados/consumidos a alto nivel	Matriz síncrono/asíncrono; mapa Request/Response con códigos de error; presupuesto de latencia
Justificación arquitectónica	Principios SOA en tabla	7 ADRs formales con contexto, decisión, alternativas y consecuencias
Riesgos	6 riesgos generales	12 riesgos técnicos clasificados por impacto/probabilidad con mitigación específica
Diagramas	Mapa de capacidades textual	Diagramas C4 Contexto + Contenedores en Structurizr DSL

10.1 Resultado esperado del APF2

Al concluir el presente avance, el equipo puede mostrar con criterio:

- Cómo se estructura la solución: seis capas verticales con responsabilidades claras.
- Cómo interactúan los servicios: matriz formal síncrono/asíncrono con presupuesto de latencia cuantificado.
- Qué contratos existen: fichas técnicas detalladas para los nueve servicios con endpoints, NFRs y riesgos.
- Por qué el diseño es coherente: siete ADRs que justifican cada decisión con sus alternativas y consecuencias.

11. Anexo: Código Structurizr DSL

El siguiente código fuente, escrito en Structurizr DSL, genera ambos diagramas C4 (Contexto y Contenedores) del sistema. Para renderizar los diagramas, copiar y pegar este código en el playground oficial de Structurizr: <https://playground.structurizr.com/>

```
workspace "Nachopps Restobar" "Plataforma SOA para gestión integral del restobar" {

    !identifiers hierarchical

    model {

        // ACTORES
        cliente = person "Cliente" "Reserva, consume y paga."
        recepcionista = person "Recepcionista" "Gestiona reservas del local."
        mesero = person "Mesero" "Toma pedidos y asigna mesas."
        cocina = person "Cocina" "Prepara ítems y actualiza estado."
        bar = person "Bar" "Prepara bebidas y actualiza estado."
        cajero = person "Cajero" "Registra pagos y arqueos."
        admin = person "Administrador" "Gestiona inventario y consulta reportes."
        gerencia = person "Gerencia" "Consulta reportes consolidados."

        // SISTEMA PRINCIPAL
        nachopps = softwareSystem "Plataforma Nachopps Restobar" "Plataforma SOA que digitaliza reservas, pedidos, cuentas, caja, inventario y reportes." {

            // CAPA DE PRESENTACIÓN
            appMesero = container "App Mesero" "App móvil para registrar pedidos y gestionar mesas." "Flutter" "WebApp"
            webCliente = container "Web Cliente (QR)" "Interfaz para reservar y consultar menú/cuenta." "React" "WebApp"
            pantallaCocina = container "Pantalla Cocina" "Muestra pedidos pendientes y permite cambiar estado." "React" "WebApp"
            pantallaBar = container "Pantalla Bar" "Muestra bebidas pendientes y permite cambiar estado." "React" "WebApp"
            posCaja = container "POS Caja" "Aplicación para registrar pagos y arqueos." "Electron + React" "WebApp"
            dashboardAdmin = container "Dashboard Administrador" "Panel para reportes, inventario y supervisión." "React" "WebApp"

            // CAPA DE APLICACIÓN
            apiGateway = container "API Gateway" "Punto único de entrada. Enruta solicitudes y valida JWT." "Kong / NGINX" "Gateway"

            // SERVICIOS DE NEGOCIO
            svcReservas = container "Servicio de Reservas" "Ciclo de vida de reservas." "Servicio de Entidad" "Service"
            svcMesas = container "Servicio de Mesas" "Estado de mesas y asignación." "Servicio de Entidad" "Service"
            svcPedidos = container "Servicio de Pedidos" "Registro y enrutamiento de ítems." "Servicio de Entidad" "Service"
            svcCuentas = container "Servicio de Cuentas" "Tickets y cierre de cuentas." "Servicio de Entidad" "Service"
            svcInventario = container "Servicio de Inventario" "Control de stock." "Servicio de Entidad" "Service"
            svcCaja = container "Servicio de Caja" "Flujo de pago y arqueos." "Servicio de Tarea" "Service"
            svcReportes = container "Servicio de Reportes" "Consolidación para dashboards." "Servicio de Tarea" "Service"

            // SERVICIOS DE UTILIDAD
            svcIdentidad = container "Servicio de Identidad" "Autenticación, RBAC, auditoría, JWT." "Servicio de Utilidad" "Support"
```

```

    svcNotificaciones = container "Servicio de Notificaciones" "Entrega notificaciones en tiempo
real." "Servicio de Utilidad" "Support"

    // MENSAJERÍA
    broker = container "Message Broker" "Distribuye eventos asíncronos (pub/sub)." "RabbitMQ"
"Broker"

    // DATOS (una BD por servicio)
    dbReservas = container "BD Reservas" "Reservas y disponibilidad." "PostgreSQL"
"Database"
    dbMesas = container "BD Mesas" "Estado de mesas." "PostgreSQL" "Database"
    dbPedidos = container "BD Pedidos" "Pedidos e ítems." "PostgreSQL" "Database"
    dbCuentas = container "BD Cuentas" "Cuentas y tickets." "PostgreSQL" "Database"
    dbInventario = container "BD Inventario" "Productos y stock." "PostgreSQL" "Database"
    dbCaja = container "BD Caja" "Movimientos y arqueos." "PostgreSQL" "Database"
    dbReportes = container "BD Reportes" "Snapshots agregados." "PostgreSQL" "Database"
    dbIdentidad = container "BD Identidad" "Usuarios, roles y tokens." "PostgreSQL" "Database"
    dbNotif = container "BD Notificaciones" "Mensajes y estado de envío." "PostgreSQL"
"Database"
    }

    // SISTEMAS EXTERNOS
    pasarelaPago = softwareSystem "Pasarela de Pagos" "Procesa pagos con tarjeta/QR." "External"
    sunat = softwareSystem "SUNAT" "Emisión de comprobantes electrónicos."
"External"
    gatewaySMS = softwareSystem "Gateway SMS/WhatsApp" "Envío de recordatorios al cliente."
"External"

    // ACTORES → APPS
    cliente -> nachopps.webCliente "Reserva y consulta menú/cuenta"
    recepcionista -> nachopps.webCliente "Gestiona reservas"
    mesero -> nachopps.appMesero "Toma pedidos y asigna mesas"
    cocina -> nachopps.pantallaCocina "Consulta y actualiza pedidos"
    bar -> nachopps.pantallaBar "Consulta y actualiza bebidas"
    cajero -> nachopps.posCaja "Registra pagos y arqueos"
    admin -> nachopps.dashboardAdmin "Gestiona inventario y reportes"
    gerencia -> nachopps.dashboardAdmin "Consulta reportes"

    // APPS → GATEWAY
    nachopps.appMesero -> nachopps.apiGateway "Llama API" "HTTPS/JSON"
    nachopps.webCliente -> nachopps.apiGateway "Llama API" "HTTPS/JSON"
    nachopps.pantallaCocina -> nachopps.apiGateway "Llama API" "HTTPS/JSON"
    nachopps.pantallaBar -> nachopps.apiGateway "Llama API" "HTTPS/JSON"
    nachopps.posCaja -> nachopps.apiGateway "Llama API" "HTTPS/JSON"
    nachopps.dashboardAdmin -> nachopps.apiGateway "Llama API" "HTTPS/JSON"

    // GATEWAY → SERVICIOS
    nachopps.apiGateway -> nachopps.svcIdentidad "Valida JWT y autoriza" "HTTPS/REST"
    nachopps.apiGateway -> nachopps.svcReservas "Enruta operaciones" "HTTPS/REST"
    nachopps.apiGateway -> nachopps.svcMesas "Enruta operaciones" "HTTPS/REST"
    nachopps.apiGateway -> nachopps.svcPedidos "Enruta operaciones" "HTTPS/REST"
    nachopps.apiGateway -> nachopps.svcCuentas "Enruta operaciones" "HTTPS/REST"
    nachopps.apiGateway -> nachopps.svcInventario "Enruta operaciones" "HTTPS/REST"
    nachopps.apiGateway -> nachopps.svcCaja "Enruta operaciones" "HTTPS/REST"
    nachopps.apiGateway -> nachopps.svcReportes "Enruta consultas" "HTTPS/REST"

    // LLAMADAS SÍNCRONAS ENTRE SERVICIOS
    nachopps.svcPedidos -> nachopps.svcInventario "GET /inventario/{itemId} + POST
/inventario/reserva" "HTTPS/REST"
    nachopps.svcCuentas -> nachopps.svcPedidos "GET /pedidos?mesa={id}" "HTTPS/REST"
    nachopps.svcCaja -> nachopps.svcCuentas "GET /cuentas/{id}/ticket" "HTTPS/REST"

    // EVENTOS PUBLICADOS
    nachopps.svcReservas -> nachopps.broker "Publica ReservaCreada, ReservaCancelada" "AMQP"
    nachopps.svcMesas -> nachopps.broker "Publica MesaAsignada, MesaLiberada" "AMQP"

```

```

nachopps.svcPedidos      -> nachopps.broker "Publica PedidoCreado, PedidoListo" "AMQP"
nachopps.svcCuentas     -> nachopps.broker "Publica CuentaCerrada, TicketGenerado" "AMQP"
nachopps.svcCaja        -> nachopps.broker "Publica PagoRegistrado, ArqueoRealizado" "AMQP"
nachopps.svcInventario  -> nachopps.broker "Publica StockBajo" "AMQP"
nachopps.svcIdentidad   -> nachopps.broker "Publica UsuarioAutenticado" "AMQP"

// EVENTOS CONSUMIDOS
nachopps.broker -> nachopps.svcMesas      "Consume ReservaCreada" "AMQP"
nachopps.broker -> nachopps.svcCuentas    "Consume PedidoListo" "AMQP"
nachopps.broker -> nachopps.svcCaja       "Consume CuentaCerrada" "AMQP"
nachopps.broker -> nachopps.svcInventario "Consume CuentaCerrada" "AMQP"
nachopps.broker -> nachopps.svcReportes   "Consume PagoRegistrado, CuentaCerrada,
StockDescontado" "AMQP"
nachopps.broker -> nachopps.svcNotificaciones "Consume y enruta todos los eventos" "AMQP"

// SERVICIOS → BD PROPIA
nachopps.svcReservas    -> nachopps.dbReservas "Lee/escribe" "JDBC"
nachopps.svcMesas       -> nachopps.dbMesas    "Lee/escribe" "JDBC"
nachopps.svcPedidos     -> nachopps.dbPedidos  "Lee/escribe" "JDBC"
nachopps.svcCuentas     -> nachopps.dbCuentas  "Lee/escribe" "JDBC"
nachopps.svcInventario  -> nachopps.dbInventario "Lee/escribe" "JDBC"
nachopps.svcCaja        -> nachopps.dbCaja     "Lee/escribe" "JDBC"
nachopps.svcReportes    -> nachopps.dbReportes "Lee/escribe" "JDBC"
nachopps.svcIdentidad   -> nachopps.dbIdentidad "Lee/escribe" "JDBC"
nachopps.svcNotificaciones -> nachopps.dbNotif  "Lee/escribe" "JDBC"

// INTEGRACIONES EXTERNAS
nachopps.svcCaja        -> pasarelaPago "Procesa pago" "HTTPS/REST"
nachopps.svcCuentas     -> sunat        "Emite comprobante electrónico" "HTTPS/SOAP"
nachopps.svcNotificaciones -> gatewaySMS "Envía SMS/WhatsApp" "HTTPS/REST"
}

views {
  systemContext nachopps "Contexto" {
    include *
    autoLayout lr
  }
  container nachopps "Contenedores" {
    include *
    autoLayout tb 400 300
  }
  styles {
    element "Person" { shape Person background #1168bd color #ffffff }
    element "Software System" { background #1168bd color #ffffff }
    element "External" { background #999999 color #ffffff }
    element "Container" { background #438dd5 color #ffffff }
    element "WebApp" { background #438dd5 color #ffffff }
    element "Gateway" { background #438dd5 color #ffffff }
    element "Service" { background #438dd5 color #ffffff }
    element "Support" { background #438dd5 color #ffffff }
    element "Broker" { shape Pipe background #438dd5 color #ffffff }
    element "Database" { shape Cylinder background #438dd5 color #ffffff }
  }
  theme default
}
}

```